ICPC Team Reference

August 17, 2026

Contents

1. Geometry

1.1. Circles

1.1.1. Circle

Circle.cpp

```
#include "../../core.h"
#include "../Primitives/Point.cpp"
#include "Sector.cpp"

template <typename T>
struct Circle {
    Point<T> c; // Center
    T r;        // Radius
    Circle() {}
    Circle(Point<T> _c, T _r) : c(_c), r(_r) {}

    // Circle from 2 points (segment as diameter)
    Circle(Point<T> p1, Point<T> p2) {
        c = (p1 + p2) / 2.0;
        r = c.distance(p1);
    }

    // Circle from 3 points (Circumcircle)
    Circle(Point<T> A, Point<T> B, Point<T> C) {
        double d = 2.0 * (A.x * (B.y - C.y) + B.x *
    (C.y - A.y) + C.x * (A.y - B.y));
        if (abs(d) < 1e-9) { c = A; r = 0; return; }

        double A2 = A.x * A.x + A.y * A.y;
        double B2 = B.x * B.x + B.y * B.y;
        double C2 = C.x * C.x + C.y * C.y;

        double cx = (A2 * (B.y - C.y) + B2 * (C.y -
    A.y) + C2 * (A.y - B.y)) / d;
        double cy = (A2 * (C.x - B.x) + B2 * (A.x -
    C.x) + C2 * (B.x - A.x)) / d;

        c = Point<T>(cx, cy);
        r = c.distance(A);
    }

    // Geometric Transformations
    void translate(const Point<T>& v) {
        c = c + v;
    }
    void scale(const Point<T>& center, double
    factor) {
        c = center + (c - center) * factor;
        r = r * factor;
    }
    void rotate(const Point<T>& center, double
    angle) {
        c = c.rotateAbout(center, angle);
    }

    // Point Inclusion
    bool onBoundary(const Point<T>& p) const {
        return abs(c.distance(p) - r) <= eps;
    }
    bool strictlyInside(const Point<T>& p) const {
        return c.distance(p) < r - eps;
    }
    bool contains(const Point<T>& p) const {
        return c.distance(p) <= r + eps;
    }
    // Splits the circle by an infinite line AB into
    two Sectors
    // Returns {Left Sector, Right Sector} with
    respect to the directed line AB
    pair<Sector<T>, Sector<T>> split(const Point<T>&
    a, const Point<T>& b) const {
        // Find line intersections
        double A = (b - a).dot(b - a);
        double B = 2 * (b - a).dot(a - c);
        double C = (a - c).dot(a - c) - r * r;

        double det = sqrt(max(0.0, B * B - 4 * A *
    C));
        double t1 = (-B + det) / (2 * A);
        double t2 = (-B - det) / (2 * A);

        Point<T> i1 = a + (b - a) * t1;
        Point<T> i2 = a + (b - a) * t2;

        // Ensure i1 is the first intersection along
    the line AB and i2 is the second
        if ((i1 - a).dot(b - a) > (i2 - a).dot(b -
    a)) swap(i1, i2);

        // Left side is counter-clockwise from i2 to
    i1
        // Right side is counter-clockwise from i1
    to i2
        return {Sector<T>(c, i2, i1), Sector<T>(c,
    i1, i2)};
    }

};

using circle = Circle<double>;
```

1.1.1.0.1. Mathematical Properties & Rules

Crucial circle theorems required to solve angles, lengths, and intersection problems without brute-force geometry:

Basic Measurements:

- **Circumference & Area:** $C = 2\pi R$, $A = \pi R^2$.
- **Arc Length:** For a central angle θ (in radians), $L = R \times \theta$.
- **Sector Area:** The area of a pie slice with central angle θ is $A = \frac{1}{2}R^2\theta$.
- **Chord Length:** A chord formed by a central angle θ has length $2R\sin(\frac{\theta}{2})$.

Theorems of Intersecting Lines:

- **Intersecting Chords Theorem:** If two chords intersect inside a circle at point P, forming segments (A, B) and (C, D), then $P_A \times P_B = P_C \times P_D$.
- **Tangent-Secant Theorem:** If a tangent segment from exterior point P touches the circle at T, and a secant from P cuts the circle at A and B, then $PT^2 = PA \times PB$.

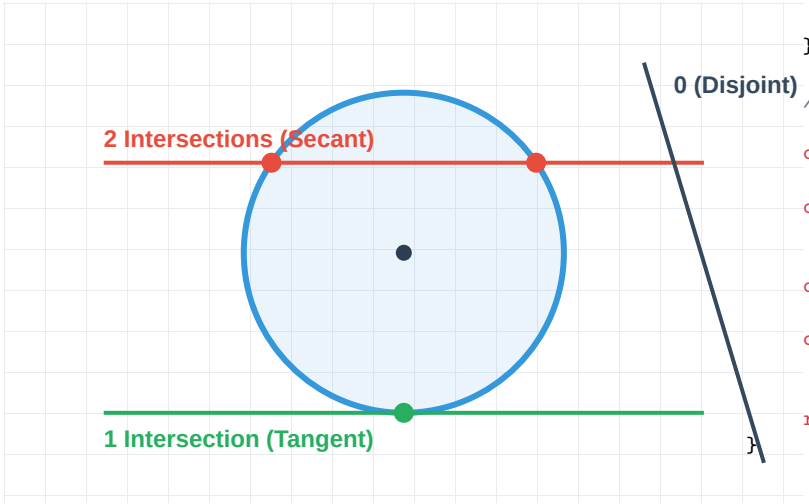
Angle Theorems:

- **Inscribed Angle Theorem:** An angle θ inscribed in a circle is exactly half of the central angle 2θ that subtends the same arc.
- **Thales's Theorem:** Any angle inscribed inside a semicircle is exactly a right angle ($90°$).

Independent functions for geometric operations on circles. Represents a Circle as a center point `cen` and radius `rad`.

1.1.1.0.2. Circle-Line Intersection

Description: Finds the exact points where an infinite line intersects a circle. The line is defined by two points P_0, P_1.




```cpp
#include "../../core.h"
#include "../Primitives/Point.cpp"

// Returns the number of intersection points (0,
    1, or 2).
// Points are stored in r1 and r2.
int circleLineIntersection(const pt& p0, const pt&
     p1, const pt& cen, double rad, pt& r1, pt& r2)
     {
    if (p0 == p1) return 0; // Invalid line

    double a = (p1 - p0).dot(p1 - p0);
    double b = 2 * (p1 - p0).dot(p0 - cen);
    double c = (p0 - cen).dot(p0 - cen) - rad *
        rad;

    double det = b * b - 4 * a * c;
    if (det < -eps) return 0; // 0 intersections

    int res = 2;
    if (abs(det) < eps) {
        det = 0;
        res = 1; // 1 intersection (Tangent)
    }

    det = sqrt(det);
    double t1 = (-b + det) / (2 * a);
    double t2 = (-b - det) / (2 * a);

    r1 = p0 + (p1 - p0) * t1;
    // Both return points with respect to the
        original segment.
    return {p1, p2};
}
```

1.1.1.0.3. Circle-Circle Intersection Area

Description: Calculates the exact overlapping area of two circles.

```cpp
#include "../../core.h"
#include "../Primitives/Point.cpp"
#include "Circle.cpp"

// Returns the intersecting area of two circles.
double circleIntersectionArea(const Circle<double
    >& c1, const Circle<double>& c2) {
    double d = c1.c.distance(c2.c);

    // 1. No overlap
    if (d >= c1.r + c2.r) return 0.0;

    // 2. One strictly inside the other
    if (d <= abs(c1.r - c2.r)) {
        double min_r = min(c1.r, c2.r);
        return acos(-1.0) * min_r * min_r;
    }

    // 3. Partial overlap (Using Law of Cosines to
         find sector angles)
    double angle1 = 2 * acos((c1.r * c1.r + d * d
        - c2.r * c2.r) / (2 * c1.r * d));
    double angle2 = 2 * acos((c2.r * c2.r + d * d
        - c1.r * c1.r) / (2 * c2.r * d));

    double area1 = 0.5 * c1.r * c1.r * (angle1 -
        sin(angle1));
    double area2 = 0.5 * c2.r * c2.r * (angle2 -
        sin(angle2));

    return area1 + area2;
}
```

1.1.1.0.4. Common Tangents of Two Circles

Description: Finding the lines that are tangent to two circles simultaneously (Internal and External tangents).

```cpp
#include "../../core.h"
#include "../Primitives/Point.cpp"
#include "Circle.cpp"
#include "../Points_and_Lines/Line.cpp"

// Adds tangents to the vector 'ans'
// inner == 1 for internal tangents, -1 for
    external tangents
void getTangents(const Circle<double>& c1, const
    Circle<double>& c2, int inner, vector<Line<
    double>>& ans) {
    double d = c1.c.distance(c2.c);
    double dr = c1.r - c2.r * inner;

    if (d < abs(dr)) return; // One circle is
        completely inside the other

    double base_angle = atan2(c2.c.y - c1.c.y, c2.
        c.x - c1.c.x);
    double offset = asin(dr / d);

    for (int sign : {-1, 1}) {
        double angle = base_angle + sign * offset;
        pt dir(cos(angle), sin(angle));

        pt p1 = c1.c + dir * (c1.r * sign * (inner
            == 1 ? -1 : 1) );
        // The tangent is perpendicular to the
            radius
        pt p2 = p1 + pt(-dir.y, dir.x);

        ans.push_back(Line<double>(p1, p2));

        // If circles touch at exactly one point,
            there is only one tangent of this type
        if (abs(d - abs(dr)) < eps) break;
    }
}
```

1.1.1.0.5. Minimum Enclosing Circle (Welzl's Algorithm)

Description: Finds the smallest mathematical circle that strictly encloses a set of N points in expected $O(N)$ time us-

ing randomization.

```cpp
#include "../../core.h"
#include "../Primitives/Point.cpp"
#include "Circle.cpp"
#include <algorithm>
#include <random>

// Helper to check if a point is inside the circle
bool inCircle(const Circle<double>& c, const Point
    <double>& p) {
    return c.c.distance(p) <= c.r + eps;
}

// Computes Minimum Enclosing Circle for a strict
    boundary of up to 3 points
Circle<double> MEC_boundary(vector<Point<double>>&
     R) {
    if (R.empty()) return Circle<double>(pt(0, 0),
         0);
    if (R.size() == 1) return Circle<double>(R[0],
         0);
    if (R.size() == 2) return Circle<double>::
        from2Points(R[0], R[1]);
    return Circle<double>::from3Points(R[0], R[1],
         R[2]);
}

// Welzl's recursive solver
Circle<double> welzl(vector<Point<double>>& P,
    vector<Point<double>> R, int n) {
    if (n == 0 || R.size() == 3) {
        return MEC_boundary(R);
    }

    // Get a random point
    Point<double> p = P[n - 1];
    Circle<double> c = welzl(P, R, n - 1);

    // If the point is inside, the MEC is still
        valid
    if (inCircle(c, p)) return c;

    // Otherwise, p MUST be on the boundary of the
        MEC
    R.push_back(p);
    return welzl(P, R, n - 1);
}

// Main function: O(N) expected time
Circle<double> minEnclosingCircle(vector<Point<
    double>> P) {
    mt19937 rng(1337);
    shuffle(P.begin(), P.end(), rng);
    return welzl(P, vector<Point<double>>(), P.
        size());
}
```

1.1.2. Sector

Sector.cpp

```cpp
#include "../../core.h"
#include "../Primitives/Point.cpp"

template <typename T>
struct Sector {
    Point<T> c; // Center of the pie
    Point<T> start; // Starting point of the arc
    Point<T> end; // Ending point of the arc
    (counter-clockwise)
    double r; // Radius

    Sector() {}
    Sector(Point<T> _c, Point<T> _start, Point<T>
    _end) : c(_c), start(_start), end(_end) {
        r = c.distance(start);
    }

    double getAngle() const {
        double angle1 = atan2(start.y - c.y, start.x
    - c.x);
        double angle2 = atan2(end.y - c.y, end.x -
    c.x);
        double diff = angle2 - angle1;
        if (diff < -eps) diff += 2 * acos(-1.0); //
    Ensure positive CCW angle
        return diff;
    }

    double area() const {
        return 0.5 * r * r * getAngle();
    }

    double perimeter() const {
        return 2 * r + r * getAngle(); // Two
    straight radii edges + curved arc
    }

    // Checks if a point lies strictly inside the
    pie slice
    bool strictlyInside(const Point<T>& p) const {
        if (c.distance(p) > r - eps) return false;
        double a = getAngle();
        double a1 = atan2(start.y - c.y, start.x -
    c.x);
        double pa = atan2(p.y - c.y, p.x - c.x);
        double diff = pa - a1;
        if (diff < -eps) diff += 2 * acos(-1.0);
        return diff < a - eps && diff > eps;
    }
};

using sector = Sector<double>;
```

1.2. Half_Planes

1.2.1. HalfPlane

HalfPlane.cpp

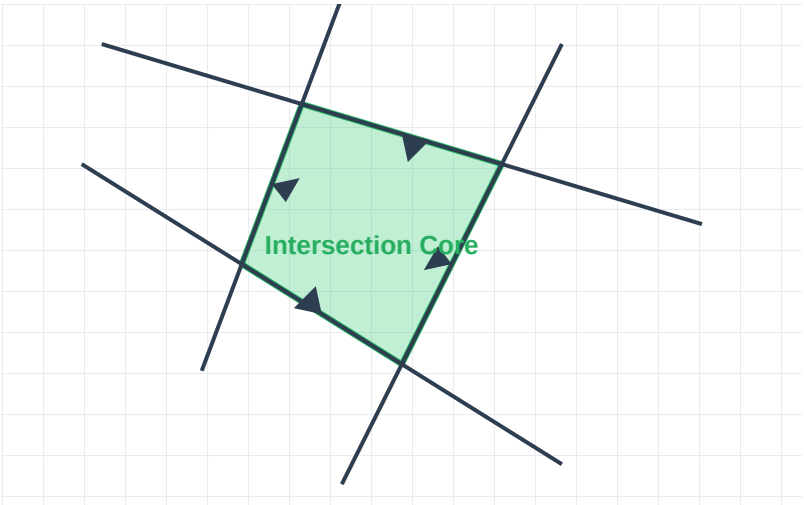
```cpp
#include "../../core.h"
#include "../Primitives/Point.cpp"

// Represents a Half-Plane defined by a line passing
    through 'p' in direction 'dir'.
// The valid region is considered to be strictly on
    the LEFT side of the directed line.
template <typename T>
struct HalfPlane {
    Point<T> p, dir;
    double angle;

    HalfPlane() {}
    HalfPlane(Point<T> a, Point<T> b) : p(a), dir(b
    - a) {
        angle = atan2(dir.y, dir.x);
    }

    // Checks if a point 'q' is strictly outside
    this half-plane (i.e. on the right side)
```

```
    bool out(const Point<T>& q) const {
        return dir.cross(q - p) < -eps;
    }

    // Sorting comparator for Half-Planes (by angle)
    bool operator<(const HalfPlane& other) const {
        return angle < other.angle;
    }
};

using hp = HalfPlane<double>;
```

1.2.1.0.1. $O(N \log N)$ Half-Plane Intersection

Description: Given N directed lines, each defining a half-plane (the left side of the line), this algorithm computes their intersection. The intersection of half-planes is always a **convex polygon** (or empty/unbounded).

- **Core Idea:** Sort the half-planes by their polar angle. Maintain a double-ended queue (`deque`) of half-planes that form the convex boundary.
- **Application:** Finding the core of a polygon, checking if a circle of radius R can fit inside a polygon (shift edges inward by R, then run HPI).




```
#include "../../core.h"
#include "../Primitives/Point.cpp"
#include "HalfPlane.cpp"
#include "../Polygons/Polygon.cpp"

// Returns the exact intersection point of the
    boundaries of two half-planes.
pt hpIntersection(const hp& a, const hp& b) {
    double cross = a.dir.cross(b.dir);
    if (abs(cross) < eps) return pt(1e18, 1e18);
        // Parallel
    double t = (b.p - a.p).cross(b.dir) / cross;
    return a.p + (a.dir * t);
}

// O ( N log N )
// Returns the convex polygon formed by the
    intersection of the half-planes.
poly halfPlaneIntersection(vector<hp>& H) {
    // 1. Add bounding box half-planes to ensure
        the intersection is bounded
    double BBOX = 1e9;
    H.push_back(hp(pt(-BBOX, -BBOX), pt(BBOX, -
        BBOX)));
    H.push_back(hp(pt(BBOX, -BBOX), pt(BBOX, BBOX)
        ));
    H.push_back(hp(pt(BBOX, BBOX), pt(-BBOX, BBOX)
        ));
    H.push_back(hp(pt(-BBOX, BBOX), pt(-BBOX, -
        BBOX)));

    // 2. Sort half-planes by angle
    sort(H.begin(), H.end());

    // 3. Remove parallel half-planes (keep the
        one that is strictly "innermost")
    vector<hp> C;
    for (int i = 0; i < H.size(); i++) {
        if (!C.empty() && abs(H[i].angle - C.back
            ().angle) < eps) {
            if (H[i].out(C.back().p)) C.back() = H
                [i];
        } else {
            C.push_back(H[i]);
        }
    }

    // 4. Deque logic
    deque<hp> dq;
    deque<pt> p; // intersection points of
        consecutive half-planes in deque

    for (int i = 0; i < C.size(); i++) {
        // Pop from back if the intersection of
            the last two is outside the current
            half-plane
        while (dq.size() >= 2 && C[i].out(p.back()
            )) {
            dq.pop_back();
            p.pop_back();
        }
        // Pop from front if the intersection of
            the first two is outside the current
            half-plane
        while (dq.size() >= 2 && C[i].out(p.front
            ())) {
            dq.pop_front();
            p.pop_front();
        }

        // Add current half plane
        dq.push_back(C[i]);
        if (dq.size() >= 2) {
            p.push_back(hpIntersection(dq[dq.size
                () - 2], dq.back()));
        }
    }

    // 5. Final cleanup for the closing edge
    while (dq.size() >= 3 && dq.front().out(p.back
        ())) {
        dq.pop_back();
        p.pop_back();
    }
    while (dq.size() >= 3 && dq.back().out(p.front
        ())) {
        dq.pop_front();
        p.pop_front();
    }

    // 6. Form the final polygon
    if (dq.size() < 3) return poly(); // Empty
        intersection

    poly res;
    p.push_back(hpIntersection(dq.back(), dq.front
        ()));
    for (auto point : p) res.add_point(point);

    return res;
}
```